

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>TIET Library — Seat Occupancy</title>

<link
href="https://fonts.googleapis.com/css2?family=DM+Mono:wght@400;500&family=Outfit:wght@300;400;600;700;900&display=swap" rel="stylesheet">

<style>

:root {

  --bg: #f0ede8;

  --paper: #faf8f5;

  --ink: #1a1714;

  --muted: #8a8580;

  --green: #2d6a4f;

  --green-bg: #d8f3dc;

  --amber: #b45309;

  --amber-bg: #fef3c7;

  --red: #9b1c1c;

  --red-bg: #fee2e2;

  --border: #ddd9d3;

  --shadow: rgba(26,23,20,0.08);

}

* { margin: 0; padding: 0; box-sizing: border-box; }

body {

  background: var(--bg);
```

```
color: var(--ink);

font-family: 'Outfit', sans-serif;

min-height: 100vh;

}
```

```
/* Subtle noise texture */
```

```
body::after {

  content: "";

  position: fixed;

  inset: 0;

  opacity: 0.025;

  background-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 256 256'
xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter id='noise'%3E%3CfeTurbulence
type='fractalNoise' baseFrequency='0.9' numOctaves='4'
stitchTiles='stitch'%3E%3C/filter%3E%3Crect width='100%25' height='100%25'
filter='url(%23noise)'/>%3E%3C/svg%3E");

  pointer-events: none;

  z-index: 999;

}
```

```
/* ---- HEADER ---- */
```

```
header {

  background: var(--ink);

  color: var(--bg);

  padding: 0 40px;

  display: flex;

  align-items: center;

  justify-content: space-between;

  gap: 20px;
```

```
flex-wrap: wrap;
min-height: 72px;
}
```

```
.header-left {
display: flex;
align-items: center;
gap: 16px;
}
```

```
.logo-badge {
background: #f7c948;
color: var(--ink);
font-weight: 900;
font-size: 13px;
padding: 5px 10px;
border-radius: 6px;
letter-spacing: 0.5px;
}
```

```
.header-title {
font-size: 15px;
font-weight: 600;
letter-spacing: 0.2px;
}
```

```
.header-sub {
font-size: 11px;
```

```
color: rgba(240,237,232,0.45);
font-family: 'DM Mono', monospace;
margin-top: 1px;
}
```

```
.live-badge {
  display: flex;
  align-items: center;
  gap: 7px;
  font-family: 'DM Mono', monospace;
  font-size: 11px;
  color: rgba(240,237,232,0.6);
  letter-spacing: 1px;
}
```

```
.live-dot {
  width: 8px; height: 8px;
  background: #4ade80;
  border-radius: 50%;
  animation: pulse 2s infinite;
}
```

```
@keyframes pulse {
  0%, 100% { opacity: 1; box-shadow: 0 0 0 rgba(74,222,128,0.5); }
  50% { opacity: 0.7; box-shadow: 0 0 5px rgba(74,222,128,0); }
}
```

```
/* ---- MAIN ---- */
```

```
main {  
  max-width: 1100px;  
  margin: 0 auto;  
  padding: 40px 24px 60px;  
}
```

```
/* ---- SUMMARY STRIP ---- */
```

```
.summary-strip {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 16px;  
  margin-bottom: 40px;  
}
```

```
.summary-card {  
  background: var(--paper);  
  border: 1px solid var(--border);  
  border-radius: 14px;  
  padding: 22px 24px;  
  box-shadow: 0 2px 8px var(--shadow);  
  display: flex;  
  flex-direction: column;  
  gap: 4px;  
  position: relative;  
  overflow: hidden;  
}
```

```
.summary-card::before {
```

```
content: "";
position: absolute;
top: 0; left: 0; right: 0;
height: 3px;
}
```

```
.card-total::before { background: var(--ink); }
.card-free::before { background: var(--green); }
.card-occupied::before { background: var(--red); }
```

```
.summary-label {
font-family: 'DM Mono', monospace;
font-size: 10px;
letter-spacing: 2px;
text-transform: uppercase;
color: var(--muted);
}
```

```
.summary-num {
font-size: 42px;
font-weight: 900;
line-height: 1;
letter-spacing: -2px;
margin-top: 4px;
}
```

```
.card-total .summary-num { color: var(--ink); }
.card-free .summary-num { color: var(--green); }
```

```
.card-occupied .summary-num { color: var(--red); }
```

```
.summary-sub {  
  font-size: 12px;  
  color: var(--muted);  
  margin-top: 2px;  
}
```

```
/* ---- SECTION TITLE ---- */
```

```
.section-heading {  
  font-size: 11px;  
  font-family: 'DM Mono', monospace;  
  letter-spacing: 3px;  
  text-transform: uppercase;  
  color: var(--muted);  
  margin-bottom: 16px;  
  display: flex;  
  align-items: center;  
  gap: 12px;  
}
```

```
.section-heading::after {  
  content: " ";  
  flex: 1;  
  height: 1px;  
  background: var(--border);  
}
```

```
/* ---- FLOOR CARDS ---- */
```

```
.floors-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));  
  gap: 18px;  
  margin-bottom: 48px;  
}
```

```
.floor-card {  
  background: var(--paper);  
  border: 1px solid var(--border);  
  border-radius: 16px;  
  padding: 24px;  
  box-shadow: 0 2px 10px var(--shadow);  
  transition: transform 0.2s, box-shadow 0.2s;  
  animation: slideIn 0.4s ease both;  
}
```

```
.floor-card:hover {  
  transform: translateY(-3px);  
  box-shadow: 0 8px 24px var(--shadow);  
}
```

```
@keyframes slideIn {  
  from { opacity: 0; transform: translateY(20px); }  
  to { opacity: 1; transform: translateY(0); }  
}
```



```
.floor-card:nth-child(1) { animation-delay: 0.05s; }  
.floor-card:nth-child(2) { animation-delay: 0.1s; }  
.floor-card:nth-child(3) { animation-delay: 0.15s; }  
.floor-card:nth-child(4) { animation-delay: 0.2s; }
```

```
.floor-top {  
  display: flex;  
  justify-content: space-between;  
  align-items: flex-start;  
  margin-bottom: 18px;  
}
```

```
.floor-name {  
  font-size: 20px;  
  font-weight: 700;  
  line-height: 1.1;  
}
```

```
.floor-desc {  
  font-size: 11px;  
  color: var(--muted);  
  margin-top: 3px;  
  font-family: 'DM Mono', monospace;  
}
```

```
.status-pill {  
  font-size: 11px;  
  font-weight: 600;
```

```
padding: 4px 12px;
border-radius: 100px;
letter-spacing: 0.5px;
white-space: nowrap;
}
```

```
.status-available { background: var(--green-bg); color: var(--green); }
.status-filling { background: var(--amber-bg); color: var(--amber); }
.status-full { background: var(--red-bg); color: var(--red); }
```

```
/* Bar */
```

```
.bar-wrap {
  height: 8px;
  background: var(--bg);
  border-radius: 100px;
  overflow: hidden;
  margin-bottom: 12px;
  border: 1px solid var(--border);
}
```

```
.bar-fill {
  height: 100%;
  border-radius: 100px;
  transition: width 1s cubic-bezier(0.4,0,0.2,1);
}
```

```
.fill-low { background: linear-gradient(90deg, #52b788, #2d6a4f); }
.fill-mid { background: linear-gradient(90deg, #f59e0b, #d97706); }
```

```
.fill-high { background: linear-gradient(90deg, #f87171, #dc2626); }
```

```
.floor-stats {  
  display: flex;  
  justify-content: space-between;  
  font-family: 'DM Mono', monospace;  
  font-size: 11px;  
  color: var(--muted);  
}
```

```
.stat-free { color: var(--green); font-weight: 500; }  
.stat-occ { color: var(--red); font-weight: 500; }
```

```
/* Seat grid visual */
```

```
.seat-grid {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 5px;  
  margin-top: 16px;  
  padding-top: 16px;  
  border-top: 1px solid var(--border);  
}
```

```
.seat {  
  width: 18px; height: 18px;  
  border-radius: 4px;  
  cursor: pointer;  
  transition: transform 0.15s;
```

```
title: ";  
}
```

```
.seat:hover { transform: scale(1.3); }  
.seat-free { background: var(--green-bg); border: 1.5px solid #95d5b2; }  
.seat-occ { background: #f8d7da; border: 1.5px solid #f5a5a8; }
```

```
/* ---- EDIT SEATS ---- */
```

```
.edit-row {  
  display: flex;  
  align-items: center;  
  gap: 8px;  
  margin-top: 14px;  
  padding-top: 14px;  
  border-top: 1px dashed var(--border);  
}
```

```
.edit-label {  
  font-family: 'DM Mono', monospace;  
  font-size: 10px;  
  color: var(--muted);  
  letter-spacing: 1px;  
  text-transform: uppercase;  
  flex-shrink: 0;  
}
```

```
.edit-input {  
  width: 64px;
```

```
padding: 5px 10px;
border: 1.5px solid var(--border);
border-radius: 8px;
background: var(--bg);
font-family: 'DM Mono', monospace;
font-size: 13px;
font-weight: 500;
color: var(--ink);
outline: none;
text-align: center;
transition: border-color 0.15s;
}
```

```
.edit-input:focus { border-color: var(--ink); }
```

```
.btn-edit-apply {
padding: 5px 14px;
border-radius: 8px;
border: none;
background: var(--ink);
color: var(--bg);
font-family: 'Outfit', sans-serif;
font-size: 12px;
font-weight: 600;
cursor: pointer;
transition: background 0.15s;
letter-spacing: 0.3px;
}
```

```
.btn-edit-apply:hover { background: #3a3530; }
```

```
.edit-hint {  
  font-family: 'DM Mono', monospace;  
  font-size: 10px;  
  color: var(--muted);  
  margin-left: auto;  
}
```

```
/* ---- CONTROLS ---- */
```

```
.controls-bar {  
  background: var(--paper);  
  border: 1px solid var(--border);  
  border-radius: 14px;  
  padding: 20px 24px;  
  margin-bottom: 40px;  
  display: flex;  
  align-items: center;  
  gap: 16px;  
  flex-wrap: wrap;  
}
```

```
.controls-label {  
  font-family: 'DM Mono', monospace;  
  font-size: 11px;  
  color: var(--muted);  
  letter-spacing: 1px;
```

```
text-transform: uppercase;
margin-right: 4px;
}
```

```
.btn-alloc, .btn-release, .btn-reset {
padding: 9px 20px;
border-radius: 100px;
border: none;
font-family: 'Outfit', sans-serif;
font-size: 13px;
font-weight: 600;
cursor: pointer;
transition: all 0.15s;
letter-spacing: 0.3px;
}
```

```
.btn-alloc { background: var(--ink); color: var(--bg); }
.btn-alloc:hover { background: #3a3530; }
```

```
.btn-release { background: var(--green-bg); color: var(--green); border: 1px solid
#95d5b2; }
```

```
.btn-release:hover { background: #b7e4c7; }
```

```
.btn-reset { background: transparent; color: var(--muted); border: 1px solid var(--
border); }
```

```
.btn-reset:hover { color: var(--ink); border-color: var(--ink); }
```

```
select.floor-select {
padding: 9px 14px;
```

```
border-radius: 100px;
border: 1px solid var(--border);
background: var(--bg);
font-family: 'Outfit', sans-serif;
font-size: 13px;
color: var(--ink);
cursor: pointer;
outline: none;
}
```

```
/* ---- LOG ---- */
```

```
.log-wrap {
background: var(--ink);
border-radius: 14px;
padding: 20px 24px;
max-height: 200px;
overflow-y: auto;
}
```

```
.log-line {
font-family: 'DM Mono', monospace;
font-size: 12px;
line-height: 2;
color: rgba(240,237,232,0.55);
animation: fadein 0.3s ease;
}
```

```
.log-line .ts { color: #f7c948; }
```



```
.log-line .action-in { color: #4ade80; }  
.log-line .action-out { color: #f87171; }  
.log-line .action-reset { color: #93c5fd; }
```

```
@keyframes fadein { from { opacity:0 } to { opacity:1 } }
```

```
/* Scrollbar */
```

```
.log-wrap::-webkit-scrollbar { width: 4px; }  
.log-wrap::-webkit-scrollbar-track { background: transparent; }  
.log-wrap::-webkit-scrollbar-thumb { background: #3a3530; border-radius: 4px; }
```

```
/* ---- LAST UPDATED ---- */
```

```
.last-updated {  
  text-align: center;  
  font-family: 'DM Mono', monospace;  
  font-size: 10px;  
  color: var(--muted);  
  letter-spacing: 1px;  
  margin-top: 32px;  
}
```

```
@media (max-width: 600px) {  
  header { padding: 14px 20px; }  
  main { padding: 24px 16px 40px; }  
  .summary-strip { grid-template-columns: 1fr 1fr; }  
  .summary-strip .card-total { grid-column: 1 / -1; }  
  .floors-grid { grid-template-columns: 1fr; }  
}
```

```
</style>

</head>

<body>

<header>

  <div class="header-left">

    <div class="logo-badge">TIET</div>

    <div>

      <div class="header-title">Library Seat Occupancy System</div>

      <div class="header-sub">UCS310 · DBMS PROJECT · 2025–26</div>

    </div>

  </div>

  <div class="live-badge">

    <div class="live-dot"></div>

    LIVE

  </div>

</header>

<main>

<!-- SUMMARY -->

<div class="summary-strip">

  <div class="summary-card card-total">

    <div class="summary-label">Total Seats</div>

    <div class="summary-num" id="total-num">120</div>

    <div class="summary-sub">across 4 floors</div>

  </div>

  <div class="summary-card card-free">
```

```
<div class="summary-label">Available</div>

<div class="summary-num" id="free-num">—</div>

<div class="summary-sub">seats free right now</div>

</div>

<div class="summary-card card-occupied">

  <div class="summary-label">Occupied</div>

  <div class="summary-num" id="occ-num">—</div>

  <div class="summary-sub">students inside</div>

</div>

</div>
```

```
<!-- FLOORS -->

<div class="section-heading">Floor-wise Breakdown</div>

<div class="floors-grid" id="floors-grid"></div>
```

```
<!-- CONTROLS -->

<div class="section-heading">Simulate Entry / Exit</div>

<div class="controls-bar">

  <span class="controls-label">Floor</span>

  <select class="floor-select" id="floor-select"></select>

  <button class="btn-alloc" onclick="allocate()">→ Student Entry</button>

  <button class="btn-release" onclick="release()">← Student Exit</button>

  <button class="btn-reset" onclick="resetAll()">↺ Reset</button>

</div>
```

```
<!-- LOG -->

<div class="section-heading">Transaction Log</div>

<div class="log-wrap" id="log"></div>
```

```
<div class="last-updated" id="last-upd"></div>
```

```
</main>
```

```
<script>
```

```
const floors = [
```

```
  { id: 1, name: "Ground Floor", desc: "GENERAL READING · PERIODICALS", total: 30 },
```

```
  { id: 2, name: "First Floor", desc: "QUIET ZONE · REFERENCE SECTION", total: 30 },
```

```
  { id: 3, name: "Second Floor", desc: "DIGITAL LAB · COMPUTER ACCESS", total: 30 },
```

```
  { id: 4, name: "Third Floor", desc: "RESEARCH WING · GROUP TABLES", total: 30 },
```

```
];
```

```
// State
```

```
const state = {};
```

```
floors.forEach(f => { state[f.id] = { occupied: Math.floor(Math.random() * (f.total * 0.6)) };  
});
```

```
function pct(f) { return state[f.id].occupied / f.total; }
```

```
function free(f) { return f.total - state[f.id].occupied; }
```

```
function statusLabel(p) {
```

```
  if (p >= 0.9) return { text: "FULL", cls: "status-full" };
```

```
  if (p >= 0.6) return { text: "FILLING FAST", cls: "status-filling" };  
  return { text: "AVAILABLE", cls: "status-available" };  
}
```

```
function barClass(p) {
```

```
if (p >= 0.9) return "fill-high";  
if (p >= 0.6) return "fill-mid";  
return "fill-low";  
}
```

```
function renderFloors() {  
  const grid = document.getElementById("floors-grid");  
  let totalFree = 0, totalOcc = 0, totalAll = 0;  
  
  floors.forEach(f => {  
    totalAll += f.total;  
    totalFree += free(f);  
    totalOcc += state[f.id].occupied;  
  });  
  
  document.getElementById("total-num").textContent = totalAll;  
  document.getElementById("free-num").textContent = totalFree;  
  document.getElementById("occ-num").textContent = totalOcc;  
  
  grid.innerHTML = floors.map(f => {  
    const p = pct(f);  
    const st = statusLabel(p);  
    const bc = barClass(p);  
    const fr = free(f);  
    const occ = state[f.id].occupied;  
    const barW = Math.round(p * 100);  
  
    // Seat dots
```

```

let seats = "";

for (let i = 0; i < f.total; i++) {

    seats += `<div class="seat ${i < occ ? 'seat-occ' : 'seat-free'}" title="${i < occ ?
'Occupied' : 'Free'}"></div>`;
}

return `

<div class="floor-card">

    <div class="floor-top">

        <div>

            <div class="floor-name">${f.name}</div>

            <div class="floor-desc">${f.desc}</div>

        </div>

        <span class="status-pill ${st.cls}">${st.text}</span>

    </div>

    <div class="bar-wrap">

        <div class="bar-fill ${bc}" style="width:${barW}%></div>

    </div>

    <div class="floor-stats">

        <span class="stat-free">▲ ${fr} free</span>

        <span>${barW}% occupied</span>

        <span class="stat-occ">▼ ${occ} taken</span>

    </div>

    <div class="seat-grid">${seats}</div>

    <div class="edit-row">

        <span class="edit-label">Total seats</span>

        <input class="edit-input" type="number" min="1" max="500" value="${f.total}"
id="edit-${f.id}" onkeydown="if(event.key==='Enter') applyTotal(${f.id})">

        <button class="btn-edit-apply" onclick="applyTotal(${f.id})">Apply</button>

```

```

        <span class="edit-hint">max 500</span>

    </div>

</div>

`;

}).join("");
}

```

```

function renderSelect() {
    const sel = document.getElementById("floor-select");
    sel.innerHTML = floors.map(f => `<option value="${f.id}">${f.name}</option>`).join("");
}

```

```

function now() {
    return new Date().toLocaleTimeString('en-IN', { hour: '2-digit', minute: '2-digit', second: '2-digit' });
}

```

```

function log(msg, type = "") {
    const div = document.getElementById("log");
    const line = document.createElement("div");
    line.className = "log-line";
    line.innerHTML = `<span class="ts">[${now()}]</span> <span class="${type}">${msg}</span>`;
    div.prepend(line);
    document.getElementById("last-upd").textContent = `Last updated: ${now()}`;
}

```

```

function allocate() {
    const fid = parseInt(document.getElementById("floor-select").value);
}

```

```

const f = floors.find(x => x.id === fid);

if (state[fid].occupied >= f.total) {

  log(` ✗ ${f.name} — No seats available. Allocation failed.` , 'action-out');

  return;

}

state[fid].occupied++;

const seatNo = state[fid].occupied;

log(` ✓ ENTRY → ${f.name} | Seat
# ${String(fid).padStart(2,'0')}${String(seatNo).padStart(3,'0')} allocated` , 'action-in');

renderFloors();

}

function release() {

  const fid = parseInt(document.getElementById("floor-select").value);

  const f = floors.find(x => x.id === fid);

  if (state[fid].occupied <= 0) {

    log(` ✗ ${f.name} — No occupied seats to release.` , 'action-out');

    return;

  }

  const seatNo = state[fid].occupied;

  state[fid].occupied--;

  log(` ✓ EXIT ← ${f.name} | Seat
# ${String(fid).padStart(2,'0')}${String(seatNo).padStart(3,'0')} released` , 'action-out');

  renderFloors();

}

function applyTotal(fid) {

  const input = document.getElementById(`edit-${fid}`);

```



```

const val = parseInt(input.value);
if (isNaN(val) || val < 1 || val > 500) {
  log(` ✕ Invalid seat count. Enter a number between 1 and 500.`, 'action-out');
  return;
}

const f = floors.find(x => x.id === fid);
const old = f.total;
f.total = val;

// Clamp occupied to new total
if (state[fid].occupied > val) {
  state[fid].occupied = val;

  log(` ⚠️ ${f.name} — Seats reduced to ${val}. Occupancy clamped to ${val}.`, 'action-reset');
} else {
  log(` 🔄 CONFIG ${f.name} | Total seats changed ${old} → ${val}`, 'action-reset');
}

renderSelect();
renderFloors();
}

```

```

function resetAll() {
  floors.forEach(f => { state[f.id].occupied = 0; });
  log('🔄 SYSTEM RESET — All seats cleared.', 'action-reset');
  renderFloors();
}

```

```

// Init
renderSelect();

```

```
renderFloors();
```

```
log(' ⚡ System initialized. Database connected.', 'action-in');
```

```
document.getElementById("last-upd").textContent = `Last updated: ${now()}`;
```

```
</script>
```

```
</body>
```

```
</html>
```